

Formation Python avancé et scientifique

Correction des travaux pratiques

Matthieu Falce

Mars 2020

1 Manipulation de la syntaxe python

1.1 *Fizz Buzz*

Voir le code de correction situé : `corrections/syntaxe/fizzBuzz.py`

1.2 Plus ou moins

Voir le code de correction situé : `corrections/syntaxe/plusOuMoins.py`

1.3 Slices

Voir le code de correction situé : `corrections/syntaxe/slices.py`

1.4 Magic 8 ball

Sous *Windows* il peut y avoir des soucis à l'ouverture du fichier à cause de l'encodage de caractère de cet *OS*. Il faut indiquer un encodage lors de l'ouverture du fichier dans ce cas.

Voir le code de correction situé : `corrections/syntaxe/magic8ball.py`

1.5 Comparaison de textes

Réponses :

1. on utilise la bibliothèque tierce `requests` (plus simple) ou ce qui existe dans la bibliothèque standard (meilleure portabilité)
2. on transforme la liste de mots en `set` (complexité $\mathcal{O}(n)$)
3. on transforme la liste de mots avec un `collections.Counter` (complexité $\mathcal{O}(n)$ je pense). Si l'on regroupe les textes, il faut *merger* les deux compteurs (ce sont des dictionnaires)
4. on utilise les opérations intersection des ensembles
5. on utilise un compteur en lui indiquant la limite à afficher

Voir le code de correction situé : `corrections/syntaxe/comparaisonTextesInternet.py`

1.6 C'est loooong

Réponses :

1. on utilise `time.sleep(3)` pour mettre en pause l'exécution de 3s

2. on utilise `time.time()` pour marquer le début et la fin de la zone à mesurer, on soustrait les valeurs et voilà
3. avec le décorateur on peut appeler plusieurs fois la fonction pour calculer moyenne et écart-type de la durée
4. il y a `timeit`

Voir le code de correction situé : `corrections/syntaxe/mesureTemps.py`

1.7 Analyse de complexité

Réponses :

1. on va générer des conteneurs de taille différentes, ensuite nous allons regarder le temps que met l'opération pour les différentes tailles, pour chaque type de conteneurs
2. nous allons utiliser `time.time` pour mesurer le temps d'exécution de la fonction d'intérêt. En réalité, cela est plus complexe à effectuer correctement et devrait effectuer une analyse statistique sur de nombreuses répétitions pour lisser l'utilisation de l'ordinateur
3. on est cohérent avec les complexités notées par la doc (les structures hashées *dict* et *set* ont des temps d'accès linéaires)
4. on utilise la bibliothèque tierce `matplotlib` sur nos différentes séries

Voir le code de correction situé : `corrections/syntaxe/complexite/analyseComplexite.py`

2 Fonctions avancées

Cette section comporte des exercices qui mettent en pratique les manipulations avancées de fonctions

2.1 Miam

Réponses :

1. le plus proche de la fonction est appelé en premier
2. oui on peut obtenir le même comportement, il faut indiquer le paramétriser sur le texte et la position avant ou après

Voir le code de correction situé : `corrections/fonctionsAvancees/miam.py`

2.2 Déprécié

Réponses :

1. cela dépend de ce que l'on veut faire :
 - avant le wrapper, on les stocke toutes
 - dans le wrapper, on ne stocke que les fonctions appellées

Voir le code de correction situé : `corrections/fonctionsAvancees/deprecie.py`

2.3 *Fizz Buzz* is back

Réponses :

1. globalement non, les solutions sont compliquées à comprendre et n'apportent pas de grands avantages

Voir le code de correction situé : `corrections/fonctionsAvancees/lazyFizzBuzz.py`

2.4 Pipeline grep

Réponses :

1. le flux étant par définition continue et infini, nous devons utiliser un pipeline qui tire les données, donc un pipeline de consommateurs

Voir le code de correction situé : `corrections/fonctionsAvancees/pipelineConsommateur.py`

2.5 Pipeline filtres

Réponses :

1. on utilise le **design pattern Chain Of Responsibility** qui permet d'ajouter ou d'enlever les filtres facilement

Voir le code de correction situé : `corrections/fonctionsAvancees/pipelineGenerateur.py`

2.6 ItertoolsBis

Voir le code de correction situé : `corrections/fonctionsAvancees/itertoolsBis.py`

2.7 Pareisseux

Réponses :

1. pour les manipulations de données dans l'ordre, **itertools** est pertinent. Cependant, pour manipuler à l'envers il ne peut rien faire.

Voir le code de correction situé : `corrections/fonctionsAvancees/sliceParaisse.py`

2.8 C'est loooong

Voir le code de correction situé : `corrections/fonctionsAvancees/looong.py`

3 Programmation orientée objet

3.1 Formation

Voir le code de correction situé : `corrections/poo/formation.py`

3.2 Convertisseur de température

Réponses :

1. c'est une mauvaise idée de dupliquer les informations, quelle est la source de vérité ? En plus, en python, le manque d'encapsulation permet de modifier les deux attributs indépendamment, ce qui peut complètement gêner le code
2. **property** c'est la vie 😊. On peut simuler des getters et setters en les manipulant comme des attributs

Voir le code de correction situé : `corrections/poo/convertisseur.py`

4 POO avancée

Cette section comporte des exercices qui mettent en pratique les manipulations avancées d'objets

4.1 Properties

Voir le code de correction situé : `corrections/pooAvancee/property.py`

4.2 La classe, la méta-classe

Réponses :

1. le `__new__` de la métaclasse n'est pas appelé, c'est celui de la classe qui l'est
2. le `__call__` de la classe n'est pas appelé, c'est celui de la métaclasse qui l'est

Voir le code de correction situé : `corrections/pooAvancee/metaclasses.py`

4.3 Singleton

Réponses :

1. cela dépend de la façon dont on s'y prend. On peut modifier le `__new__` d'une classe ou le `__call__` d'une méta classe
2. on utilise l'attribut `metaclass` de `object`
3. c'est une question de point de vue...

Voir le code de correction situé : `corrections/pooAvancee/singletonMetaclass.py`

5 Modules

5.1 Bases de données

Réponses :

1. on peut en stocker autant que nécessaire pour la fonctionnalité métier. Dans ce cas, stocker, le nom, matière et note semble être un minimum
2. on utilise du SQL pour effectuer les recherches (filtres, accès), on peut également utiliser un *ORM object relationship manager* pour ne pas avoir à écrire de code

Voir le code de correction situé : `corrections/modules/bdd/sqlite.py`

Voir le code de correction situé : `corrections/modules/bdd/sqlalchemyImplementation.py` (pour l'utilisation d'un ORM)

5.2 Expressions régulières

Tous, trouvez les tous

Réponses :

1. on parse le fichier en utilisant une expression régulière
2. une fois parsé, on met les informations dans un `collection.Counter`

Voir le code de correction situé : `corrections/modules/regexp/versions.py`

Cherchez / trouvez

Réponses :

1. on met une option sur la lettre "h"
2. on met une contrainte sur le début et la fin du mot (par exemple, il faut que ce soit un espace ou le début / fin de ligne)
3. non, quand on commence à construire des expressions trop complexes, il peut être intéressant de développer sa propre solution en python (le langage permettant de facilement travailler les chaînes de caractères)

Voir le code de correction situé : `corrections/modules/regex/chat.py`

Identifiants

Réponses :

1. toutes ces chaînes sont structurées de la même façon
2. on peut donc utiliser une expression régulière pour capturer les différents groupes

Voir le code de correction situé : `corrections/modules/regex/extractUUID.py`

C'est valide

Réponses :

1. une adresse email doit ressembler à une forme canonique. Attention, cela n'est pas la bonne façon de faire, pour être sûr que le mail existe, il faut l'envoyer, les serveurs mails se chargeront de chercher à le distribuer.
2. on peut créer une expression régulière permettant de vérifier ses informations
3. on ouvre le JSON et on extrait des groupes de capture

Voir le code de correction situé : `corrections/modules/regex/validationEmails.py`

6 Écosystème scientifique

6.1 L'addition s'il vous plaît

Réponses :

1. *cf.* cours
2. par défaut la multiplication est terme à terme
3. il faut utiliser l'opérateur `@`

Voir le code de correction situé : `corrections/scipy/additions.py`

6.2 Un vrai peintre

Voir le code de correction situé : `corrections/scipy/creationTableaux.py`

6.3 Le beau tableau

Il faut manipuler les tableaux `numpy` et modifier les réglages de `matplotlib` pour y arriver.

Voir le code de correction situé : `corrections/scipy/beauTableau.py`

6.4 Jouons avec des fonctions

Réponses :

1. on utilise `np.testing.assert_almost_equal`
2. on utilise `timeit` , les différences sont potentiellement trop faibles pour les mesurer autrement

Voir le code de correction situé : `corrections/scipy/fonctions.py`

6.5 Sinus cardinal

Voir le code de correction situé : `corrections/scipy/sinc.py`

6.6 Rien ne va plus

Voir le code de correction situé : `corrections/scipy/pi.py`

6.7 Magic 8 ball

Réponses :

1. on utilise un test du χ^2 pour tester l'uniformité du tirage

Voir le code de correction situé : `corrections/scipy/magic8ballScientifique.py`

6.8 Marche aléatoire

Voir le code de correction situé : `corrections/scipy/marcheAleatoire.py`

6.9 Je me Gausse

On utilise la notation `@` pour effectuer des multiplications matricielles.

Voir le code de correction situé : `corrections/scipy/systemeEquations.py`

6.10 Mon champion

Voir le code de correction situé : `corrections/scipy/jo.py`

6.11 On se mate une série temporelle ?

Voir le code de correction situé : `corrections/scipy/serieTemporelle.py`

6.12 Analyse Covid

Voir le code de correction situé : `corrections/scipy/Covid.ipynb`

6.13 C'est imagé !

Voir le code de correction situé : `corrections/scipy/images.ipynb`

6.14 Des renards et des lapins

Voir le code de correction situé : `corrections/scipy/lotkaVolterra.py`

6.15 Trop de bruit

Voir le code de correction situé : `corrections/scipy/fftFiltrage.py`

6.16 Simplifions

Voir le code de correction situé : `corrections/scipy/sympy.py`